

```

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WebServer.h>
#include <HTTPClient.h>
#include <Adafruit_Fingerprint.h>
#include "LittleFS.h"
#include "RTCLib.h"
#include "BluetoothSerial.h"
#include <Preferences.h>
#include <ArduinoJson.h>

// CONFIGURACIÓN DE PINES
#define RXD2 16
#define TXD2 17
// CONFIGURACIÓN DE PINES DIGITALES PUROS (SIN INTERFERENCIA DE SERIAL NI WIFI)
#define LED_ROJO 26 // Etiqueta D26 (Lado izquierdo) - Estable y libre de ruidos
#define LED_VERDE 18 // Etiqueta D18 (Lado derecho) - Digital puro, sin parpadeos
#define LED_AZUL 5 // Etiqueta D5 (Lado derecho) - Digital puro, sin parpadeos
#define BUZZER 25 // Etiqueta D25 (Lado izquierdo) - Salida digital directa de alta corriente

// CONFIGURACIÓN FIREBASE
const char* FB_BASE_URL =
"https://asistencia-93328-default-rtdb.firebaseio.com";
const char* FB_ASISTENCIA =
"https://asistencia-93328-default-rtdb.firebaseio.com/asistencia.json";

// OBJETOS
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&Serial2);
RTC_DS3231 rtc;
BluetoothSerial SerialBT;
WebServer server(80);
Preferences prefs;

// VARIABLES GLOBALES
String ssid, pass;
String ipGuardada = "0.0.0.0";
unsigned long lastSync = 0;
unsigned long btTimer = 0;
unsigned long lastWifiRetry = 0;
unsigned long lastFichadaTime = 0;
bool btActivo = true;
bool btAutenticado = false;
bool ventanaAbierta = true;
bool wifiIniciado = false;

// ENUMERACIÓN DE ALERTAS PARA EL SISTEMA
enum TipoAlerta {
    FICHADA_OK,
    FICHADA_ERROR,

```

```

    OFFLINE_ALERTA,
    ENROL_ESPERA,
    ENROL_OK,
    ENROL_ERROR
};

// PROTOTIPOS
void activarModoWifi();
void verificarConexion();
void procesarWifiBT(String msg);
void procesarFechaBT(String msg);
void realizarResetTotal();
void syncTempToFirebase();
void verificarHuellaAsistencia();
void handleGetUsers();
void handleEnrol();
void handleDelete();
int findFreeID();
bool captureStep(int step);
void mostrarFechaHoraRTC();
void notificarSistema(TipoAlerta alerta);

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\n[SISTEMA] --- INICIANDO SISTEMA INTEGRADO ---");

    // Configuración de Periféricos de Salida
    pinMode(LED_VERDE, OUTPUT);
    pinMode(LED_ROJO, OUTPUT);
    pinMode(LED_AZUL, OUTPUT);
    pinMode(BUZZER, OUTPUT);

    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_ROJO, LOW);
    digitalWrite(LED_AZUL, LOW);
    digitalWrite(BUZZER, LOW);

    WiFi.persistent(false);
    WiFi.disconnect(true);
    WiFi.mode(WIFI_OFF);
    delay(200);

    if (!LittleFS.begin(true)) Serial.println("[ERR] Fallo LittleFS");
    if (!rtc.begin()) {
        Serial.println("[ERR] Fallo RTC");
    } else {
        Serial.println("[OK] Módulo RTC DS3231 detectado.");
        mostrarFechaHoraRTC();
    }

    prefs.begin("wifi-config", true);
    ssid = prefs.getString("ssid", "Vilers-2");
    pass = prefs.getString("pass", "Pabada686");
}

```

```

ipGuardada = prefs.getString("ip_local", "0.0.0.0");
prefs.end();

Serial2.begin(57600, SERIAL_8N1, RXD2, TXD2);
finger.begin(57600);
if (finger.verifyPassword()) {
    Serial.println("[OK] Sensor de huella en línea.");
} else {
    Serial.println("[ERR] No se encontró el sensor de huella.");
}

Serial.println("[BT] Modo Configuración Activado. Esperando comandos...");
SerialBT.begin("Laggersoft");
btTimer = millis();
lastFichadaTime = millis();

server.serveStatic("/style.css", LittleFS, "/style.css");
server.serveStatic("/script.js", LittleFS, "/script.js");
server.on("/", []() {
    File file = LittleFS.open("/index.html", "r");
    if (!file) {
        Serial.println("[ERR] index.html no encontrado en LittleFS");
        server.send(404, "text/plain", "Falta index.html en memoria");
        return;
    }
    server.streamFile(file, "text/html");
    file.close();
});

server.on("/get_users", handleGetUsers);
server.on("/enrol", handleEnrol);
server.on("/delete", handleDelete);
}

void loop() {
    // 1. GESTIÓN COMUNICACIÓN BLUETOOTH
    if (btActivo) {
        if (ventanaAbierta || btAutenticado) {
            if (SerialBT.available()) {
                String msg = SerialBT.readStringUntil('\n');
                msg.trim();
                Serial.printf("[BT RX]: %s\n", msg.c_str());
                if (msg == "OBIWANKENOBI") {
                    btAutenticado = true;
                    ventanaAbierta = false;
                    SerialBT.printf("IP_LOCAL:[%s]\n", ipGuardada.c_str());
                    Serial.printf("[BT TX] Enviando IP guardada a la App: %s\n",
ipGuardada.c_str());
                } else if (msg == "CORTARBT" && btAutenticado) {
                    activarModoWifi();
                } else if (msg.startsWith("SSIDPASS:")) {
                    procesarWifiBT(msg);
                } else if (btAutenticado) {
                    if (msg.startsWith("DATETIME:")) procesarFechaBT(msg);
                }
            }
        }
    }
}

```

```

        if (msg == "RESET") realizarResetTotal();
    }
}
}
if (ventanaAbierta && (millis() - btTimer > 20000)) {
    Serial.println("[BT] Tiempo de espera agotado. Iniciando WiFi...");
    activarModoWifi();
}
}

// 2. GESTIÓN COMUNICACIÓN WIFI Y PETICIONES WEB
if (wifiIniciado) {
    verificarConexion();
    server.handleClient();

    if (WiFi.status() == WL_CONNECTED && (millis() - lastSync > 30000)) {
        syncTempToFirebase();
        lastSync = millis();
    }
} else {
    if (millis() - lastFichadaTime > 600000) {
        Serial.println("[SISTEMA] Desborde de 10 min sin fichadas. Intentando
conectar a internet para vaciar cola...");
        activarModoWifi();
        lastFichadaTime = millis();
    }
}

// 3. GESTIÓN CONSTANTE DEL SENSOR DE HUELLAS
verificarHuellaAsistencia();
}

void activarModoWifi() {
    Serial.println("[RADIO] Apagando Bluetooth de forma segura...");
    SerialBT.end();
    btActivo = false;
    Serial.printf("[RADIO] Encendiendo WiFi. Conectando a: %s\n", ssid.c_str());
    WiFi.mode(WIFI_STA);
    delay(100);
    WiFi.begin(ssid.c_str(), pass.c_str());
    lastWifiRetry = millis() + 10000;
    server.begin();
    wifiIniciado = true;
}

void verificarConexion() {
    if (WiFi.status() != WL_CONNECTED) {
        if (millis() - lastWifiRetry > 20000) {
            lastWifiRetry = millis();
            Serial.println("[WIFI] Buscando red, reintentando conexión...");
            WiFi.begin(ssid.c_str(), pass.c_str());
        }
    } else {
        static bool logConexion = false;

```

```

if (!logConexion) {
    String ipActual = WiFi.localIP().toString();
    Serial.print("[WIFI] ¡Conectado con éxito! Dirección IP: ");
    Serial.println(WiFi.localIP());

    if (ipActual != ipGuardada) {
        ipGuardada = ipActual;
        prefs.begin("wifi-config", false); // Modo escritura
        prefs.putString("ip_local", ipGuardada);
        prefs.end();
        Serial.println("[PREFERENCES] Nueva dirección IP respaldada en Flash.");
    }

    logConexion = true;
}
}
}

void procesarWifiBT(String msg) {
    int c1 = msg.indexOf('['), c2 = msg.indexOf(']');
    int c3 = msg.lastIndexOf('['), c4 = msg.lastIndexOf(']');
    if (c1 != -1 && c3 != -1) {
        ssid = msg.substring(c1 + 1, c2);
        pass = msg.substring(c3 + 1, c4);
        prefs.begin("wifi-config", false);
        prefs.putString("ssid", ssid);
        prefs.putString("pass", pass);
        prefs.end();
        Serial.println("[OK] Nuevos datos de WiFi guardados. Reiniciando placa...");
        delay(500);
        ESP.restart();
    }
}

void procesarFechaBT(String msg) {
    int start = msg.indexOf '[' + 1;
    int end = msg.lastIndexOf(']');
    String valStr = msg.substring(start, end);
    int v[6];
    int count = 0;
    int lastPos = 0;
    for (int i = 0; i <= valStr.length(); i++) {
        if (i == valStr.length() || valStr[i] == ',') {
            v[count++] = valStr.substring(lastPos, i).toInt();
            lastPos = i + 1;
            if (count == 6) break;
        }
    }
    if (count == 6) {
        rtc.adjust(DateTime(v[0], v[1], v[2], v[3], v[4], v[5]));
        Serial.println("[OK] Hora del módulo RTC sincronizada.");
    }
}
}

```

```

void realizarResetTotal() {
    Serial.println("[SISTEMA] Borrando datos de almacenamiento y memoria...");
    finger.emptyDatabase();

    File root = LittleFS.open("/");
    File file = root.openNextFile();
    while (file) {
        String fileName = String(file.name());
        if (fileName.endsWith(".csv")) {
            LittleFS.remove "/" + fileName);
        }
        file = root.openNextFile();
    }

    prefs.begin("wifi-config", false);
    prefs.clear();
    prefs.end();
    delay(500);
    ESP.restart();
}

void verificarHuellaAsistencia() {
    if (finger.getImage() == FINGERPRINT_OK) {
        lastFichadaTime = millis();

        if (finger.image2Tz() == FINGERPRINT_OK && finger.fingerFastSearch() ==
FINGERPRINT_OK) {
            DateTime now = rtc.now();
            String data = String(finger.fingerID) + "," + now.timestamp();
            Serial.printf("[ASISTENCIA] ID de Huella %d detectado correctamente.\n",
finger.fingerID);

            char filename[32];
            snprintf(filename, sizeof(filename), "/%04d_%02d_%02d.csv", now.year(),
now.month(), now.day());

            File f1 = LittleFS.open(filename, FILE_APPEND);
            if (f1) {
                f1.println(data);
                f1.close();
                Serial.printf("[LOCAL] Guardado en ráfaga: %s\n", filename);
            }

            if (WiFi.status() == WL_CONNECTED) {
                notificarSistema(FICHADA_OK);
            } else {
                notificarSistema(OFFLINE_ALERTA);
            }
        } else {
            Serial.println("[ASISTENCIA] Advertencia: Huella dactilar no
registrada.");
            notificarSistema(FICHADA_ERROR);
        }
        delay(1000);
    }
}

```

```

    }
}

void handleGetUsers() {
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    String url = String(FB_BASE_URL) + "/tbl_alumnos.json";
    Serial.printf("[WEB SERVER] GET Alumnos -> %s\n", url.c_str());
    http.begin(client, url);
    int code = http.GET();
    Serial.printf("[WEB SERVER] Firebase respondió Código HTTP: %d\n", code);
    server.send(code, "application/json", http.getString());
    http.end();
}

void handleEnrol() {
    server.setHeader("Access-Control-Allow-Origin", "*");
    String uid = server.arg("uid");
    int id = findFreeID();
    if (id == -1) {
        server.send(200, "text/plain", "No hay espacio para mas huellas");
        return;
    }

    Serial.println("[ENROL] Iniciando proceso responsivo...");

    // =====
    // PASO 1: COLOCAR EL DEDO POR PRIMERA VEZ
    // =====
    digitalWrite(LED_AZUL, HIGH); // Se enciende: Esperando que apoye el dedo

    // Bucle de espera activa: Mientras NO haya un dedo apoyado, el LED sigue
    encendido
    unsigned long timeout1 = millis();
    while (finger.getImage() != FINGERPRINT_OK) {
        if (millis() - timeout1 > 10000) { // Timeout de 10 segundos por seguridad
            server.send(200, "text/plain", "Error: Tiempo de espera agotado");
            digitalWrite(LED_AZUL, LOW);
            notificarSistema(ENROL_ERROR);
            return;
        }
    }
    delay(50); // Muestreo rápido del sensor
}

// ¡Dedo detectado! Apagamos el LED Azul inmediatamente para dar feedback
visual
digitalWrite(LED_AZUL, LOW);

// Procesar la primera captura
if (finger.image2Tz(1) != FINGERPRINT_OK) {
    server.send(200, "text/plain", "Error: No se pudo procesar la imagen 1");
    notificarSistema(ENROL_ERROR);
    return;
}

```

```

}

// =====
// PASO 2: SOLICITAR QUE LEVANTE EL DEDO
// =====
Serial.println("[ENROL] Paso 1 capturado. Esperando que levante el dedo...");

// Hacemos parpadear el LED azul rápido para indicarle físicamente que debe
RETIRAR el dedo
unsigned long waitTime = millis();
while (finger.getImage() != FINGERPRINT_NOFINGER) {
    digitalWrite(LED_AZUL, HIGH);
    delay(100);
    digitalWrite(LED_AZUL, LOW);
    delay(100);
    if (millis() - waitTime > 6000) break;
}
digitalWrite(LED_AZUL, LOW); // Asegurar que quede apagado
delay(1000);                // Pausa de confort para que se prepare para el
segundo paso

// =====
// PASO 3: COLOCAR EL DEDO PARA VERIFICAR
// =====
Serial.println("[ENROL] Coloque el mismo dedo nuevamente...");
digitalWrite(LED_AZUL, HIGH); // Se vuelve a encender: Esperando que apoye
para verificar

// Bucle de espera activa para el segundo toque
unsigned long timeout2 = millis();
while (finger.getImage() != FINGERPRINT_OK) {
    if (millis() - timeout2 > 10000) {
        server.send(200, "text/plain", "Error: Tiempo de espera agotado en
verificación");
        digitalWrite(LED_AZUL, LOW);
        notificarSistema(ENROL_ERROR);
        return;
    }
    delay(50);
}

// ¡Dedo detectado por segunda vez! Apagamos el LED Azul inmediatamente
digitalWrite(LED_AZUL, LOW);

// Procesar la segunda captura y emparejar
if (finger.image2Tz(2) == FINGERPRINT_OK && finger.createModel() ==
FINGERPRINT_OK && finger.storeModel(id) == FINGERPRINT_OK) {

    // Almacenamiento exitoso en hardware, procedemos a impactar en Firebase
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    String url = String(FB_BASE_URL) + "/tbl_alumnos/" + uid + "/huellaId.json";
    http.begin(client, url);

```



```

    int httpResponseCode = http.PUT(String(id));
    Serial.printf("[ENROL] Actualizando Firebase. Código HTTP: %d\n",
httpResponseCode);

    if (httpResponseCode == 200) {
        server.send(200, "text/plain", "OK");
        notificarSistema(ENROL_OK); // Lanza el LED Verde y el Buzzer con pitido
corto
        Serial.printf("[ENROL] Éxito absoluto. Huella asignada al ID: %d\n", id);
    } else {
        server.send(200, "text/plain", "Error Firebase");
        notificarSistema(ENROL_ERROR); // Lanza el LED Rojo y el Buzzer largo
    }
    http.end();
} else {
    // Si las huellas no coinciden o falló el modelado
    server.send(200, "text/plain", "Fallo: Las huellas no coinciden");
    notificarSistema(ENROL_ERROR); // Lanza el LED Rojo y el Buzzer largo
}
}
}

```

```

void handleDelete() {
    server.setHeader("Access-Control-Allow-Origin", "*");
    String uid = server.arg("uid");
    if (uid == "") {
        server.send(400, "text/plain", "Falta UID");
        return;
    }
    WiFiClientSecure client;
    client.setInsecure();
    HTTPClient http;
    String urlUser = String(FB_BASE_URL) + "/tbl_alumnos/" + uid + ".json";
    http.begin(client, urlUser);
    int httpCode = http.GET();
    if (httpCode == 200) {
        String payload = http.getString();
        JsonDocument doc;
        deserializeJson(doc, payload);
        int idSensor = doc["huellaId"];
        if (idSensor > 0 && idSensor <= 1000) {
            if (finger.deleteModel(idSensor) == FINGERPRINT_OK) {
                Serial.printf("[DELETE] Removido ID %d de la memoria física del
sensor\n", idSensor);
            }
        }
    }
    http.end();

    http.begin(client, urlUser);
    int deleteCode = http.sendRequest("DELETE");
    Serial.printf("[DELETE] Borrando en Firebase. Código HTTP: %d\n",
deleteCode);
    if (deleteCode == 200 || deleteCode == 204) {
        server.send(200, "text/plain", "OK");
    }
}

```

```

    // CORREGIDO: Lógica directa para encender y apagar el LED Verde
    digitalWrite(LED_VERDE, HIGH); // Enciende
    delay(400);
    digitalWrite(LED_VERDE, LOW); // Apaga

    Serial.println("[DELETE] Alumno purgado del ecosistema.");
} else {
    server.send(500, "text/plain", "Error al eliminar de Firebase");
}
http.end();
} else {
    server.send(404, "text/plain", "Usuario no encontrado");
    http.end();
}
}

bool captureStep(int step) {
    int p = -1;
    unsigned long timeout = millis();
    while (p != FINGERPRINT_OK && (millis() - timeout < 2500)) {
        p = finger.getImage();
    }
    return (p == FINGERPRINT_OK && finger.image2Tz(step) == FINGERPRINT_OK);
}

int findFreeID() {
    for (int i = 1; i <= 1000; i++) {
        if (finger.loadModel(i) != FINGERPRINT_OK) return i;
    }
    return -1;
}

void mostrarFechaHoraRTC() {
    if (!rtc.begin()) {
        Serial.println("[RTC] Error: El módulo DS3231 no está respondiendo en el bus I2C.");
        return;
    }
    DateTime ahora = rtc.now();
    Serial.println("\n--- [DIAGNÓSTICO RTC] ---");
    Serial.printf("Fecha/Hora Actual (Timestamp): %s\n",
ahora.timestamp().c_str());
    Serial.printf("Detalle: %02d/%02d/%04d %02d:%02d:%02d\n", ahora.day(),
ahora.month(), ahora.year(), ahora.hour(), ahora.minute(), ahora.second());
    if (ahora.year() < 2020) {
        Serial.println("[ALERTA RTC] La fecha es muy antigua. Es probable que la
batería CR2032 esté agotada o falte sincronización.");
    }
    Serial.println("-----\n");
}

void notificarSistema(TipoAlerta alerta) {
    switch (alerta) {

```

```

case FICHADA_OK:
    digitalWrite(LED_VERDE, HIGH); // Enciende
    digitalWrite(BUZZER, HIGH);
    delay(150);
    digitalWrite(BUZZER, LOW);
    delay(350);
    digitalWrite(LED_VERDE, LOW); // Apaga
    break;

case FICHADA_ERROR:
    digitalWrite(LED_ROJO, HIGH); // Enciende
    digitalWrite(BUZZER, HIGH);
    delay(800);
    digitalWrite(BUZZER, LOW);
    digitalWrite(LED_ROJO, LOW); // Apaga
    break;

case OFFLINE_ALERTA:
    for (int i = 0; i < 3; i++) {
        digitalWrite(LED_ROJO, HIGH); // Enciende
        digitalWrite(BUZZER, HIGH);
        delay(600);
        digitalWrite(BUZZER, LOW);
        digitalWrite(LED_ROJO, LOW); // Apaga
        if (i < 2) delay(200);
    }
    break;

case ENROL_ESPERA:
    digitalWrite(LED_AZUL, HIGH); // Enciende LED Azul fijo de espera
    break;

case ENROL_OK:
    digitalWrite(LED_VERDE, HIGH); // Enciende
    digitalWrite(BUZZER, HIGH);
    delay(150);
    digitalWrite(BUZZER, LOW);
    delay(400);
    digitalWrite(LED_VERDE, LOW); // Apaga
    break;

case ENROL_ERROR:
    digitalWrite(LED_ROJO, HIGH); // Enciende
    digitalWrite(BUZZER, HIGH);
    delay(800);
    digitalWrite(BUZZER, LOW);
    digitalWrite(LED_ROJO, LOW); // Apaga
    break;
}
}

void syncTempToFirebase() {
    DateTime now = rtc.now();

```

```

char todayFilename[32];
snprintf(todayFilename, sizeof(todayFilename), "%04d_%02d_%02d.csv",
now.year(), now.month(), now.day());

File root = LittleFS.open("/");
File file = root.openNextFile();

WiFiClientSecure client;
client.setInsecure();
HTTPClient http;

Serial.println("[FIREBASE] Iniciando ciclo de sincronización por lotes...");

while (file) {
    String fileName = String(file.name());

    if (fileName.startsWith("/")) {
        fileName = fileName.substring(1);
    }

    if (fileName.endsWith(".csv")) {
        String nodeKey = fileName;
        int dotIndex = nodeKey.lastIndexOf('.');
        if (dotIndex != -1) {
            nodeKey = nodeKey.substring(0, dotIndex);
        }

        String fullPath = "/" + fileName;
        File f = LittleFS.open(fullPath, "r");
        if (!f) {
            file = root.openNextFile();
            continue;
        }

        if (f.size() == 0) {
            f.close();
            if (fileName != String(todayFilename)) {
                LittleFS.remove(fullPath);
            }
            file = root.openNextFile();
            continue;
        }

        bool currentFileOk = true;
        Serial.printf("[FIREBASE] Enviando lote de fichadas: %s al nodo
/asistencia/%s/\n", fileName.c_str(), nodeKey.c_str());

        while (f.available()) {
            String line = f.readStringUntil('\n');
            line.trim();
            if (line.length() > 0) {
                unsigned long idVariable = 1770000000000ULL + (millis() %
9192516961ULL);
                String urlDestino = String(FB_BASE_URL) + "/asistencia/" + nodeKey +

```

```

"/" + String(idVariable) + ".json";
    http.begin(client, urlDestino);
    http.addHeader("Content-Type", "application/json");
    // SOLUCIÓN DEFINITIVA: Arma exactamente la cadena fija original que
acepta tu Firebase
    String jsonPayload = String("{\"\\"fichada\\":\\""} + line +
String("\\"}\\");

    int code = http.PUT(jsonPayload);
    if (code == 200 || code == 201) {
        Serial.printf("[FIREBASE] %s -> ID %s enviado.\n", nodeKey.c_str(),
String(idVariable).c_str());
    } else {
        Serial.printf("[ERR] Error al subir línea en nodo %s. Código: %d\n",
nodeKey.c_str(), code);
        currentFileOk = false;
    }
    http.end();
    delay(50);
}
}
f.close();
if (currentFileOk) {
    if (fileName == String(todayFilename)) {
        File fVaciar = LittleFS.open(fullPath, FILE_WRITE);
        if (fVaciar) {
            fVaciar.close();
            Serial.println("[LOCAL] Archivo de hoy sincronizado y vaciado para
nuevas fichadas.");
        }
    } else {
        LittleFS.remove(fullPath);
        Serial.printf("[LOCAL] Archivo histórico %s eliminado
definitivamente.\n", fullPath.c_str());
    }
} else {
    Serial.printf("[ALERTA] Lote %s se subió con errores parciales. Se
reintentará el lote completo.\n", fileName.c_str());
}
}
file = root.openNextFile();
}
}

```